EXPERIMENT 1 Clustering based data analytics using R/Python. (K-Means, SOM algorithms)

AIM

To implement and compare clustering algorithms – K-Means and Self-Organizing Map (SOM) – for data analytics using Python, and evaluate their performance on the Tips dataset.

PROCEDURE

1. Import required libraries: numpy, pandas, matplotlib, sklearn (KMeans, StandardScaler, silhouette_score, adjusted_rand_score), and minisom.
2. Load the Tips dataset using seaborn's load_dataset() function.
3. Select the features: total_bill, tip, and size for clustering.
4. Normalize the features using StandardScaler.
5. Apply K-Means clustering with n_clusters=3 and random_state=42.
6. Store cluster labels and centroids.
7. Visualize K-Means clusters using a scatter plot.
8. Install and import MiniSom library.
9. Initialize MiniSom with a 10x10 grid, input_len=3, sigma=1.0, learning_rate=0.5.
10. Train the SOM with 1000 iterations.
11. Assign each data point to its Best Matching Unit (BMU).
12. Visualize SOM clustering using a scatter plot.
13. Compute Silhouette Scores for both algorithms.
14. Compute Adjusted Rand Index (ARI) for both algorithms using quantile-based ground truth.
15. Compare and declare the winner based on scores.

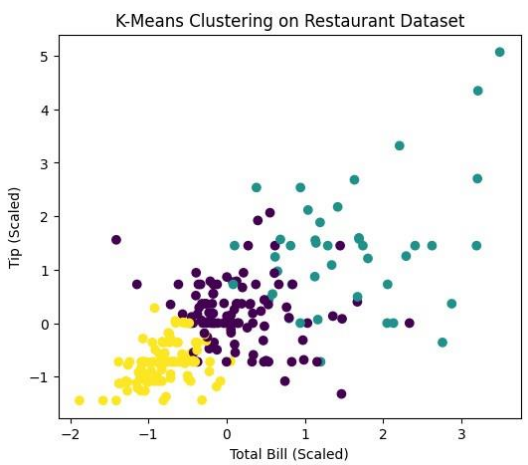
CODE

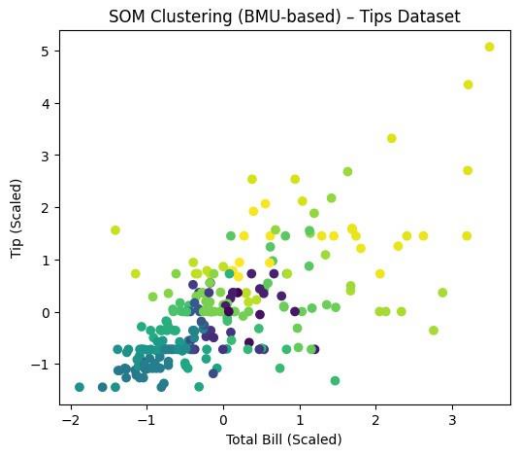
```
import numpy as np import
pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns

data = sns.load_dataset('tips') data.head()

X = data[['total_bill', 'tip', 'size']].values

from sklearn.preprocessing import StandardScaler scaler
= StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.cluster import KMeans kmeans =
KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled) labels = kmeans.labels_
centroids = kmeans.cluster_centers_

plt.figure(figsize=(6,5))
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=labels, cmap='viridis')
plt.xlabel('Total Bill (Scaled)') plt.ylabel('Tip (Scaled)')
plt.title('K-Means Clustering on Restaurant Dataset') plt.show()

!pip install minisom from
minisom import MiniSom
```

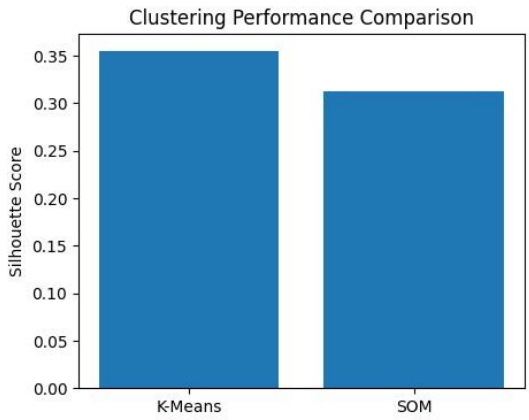
```
som = MiniSom(x=10, y=10, input_len=3, sigma=1.0, learning_rate=0.5)
som.random_weights_init(X_scaled)
som.train_random(X_scaled, num_iteration=1000)

som_labels = [som.winner(x)[0]*10 + som.winner(x)[1] for x in X_scaled]

plt.figure(figsize=(6,5)) plt.scatter(X_scaled[:,0],
X_scaled[:,1], c=som_labels) plt.xlabel('Total Bill
(Scaled)') plt.ylabel('Tip (Scaled)')
plt.title('SOM Clustering (BMU-based) - Tips Dataset') plt.show()

from sklearn.metrics import silhouette_score, adjusted_rand_score
silhouette_kmeans = silhouette_score(X_scaled, labels) silhouette_som
= silhouette_score(X_scaled, som_labels)

true_labels = pd.qcut(data['total_bill'], q=3, labels=[0, 1, 2]).astype(int).values
ari_kmeans = adjusted_rand_score(true_labels, labels) ari_som =
adjusted_rand_score(true_labels, som_labels)

print('Silhouette Score (K-Means):', silhouette_kmeans)
print('Silhouette Score (SOM):', silhouette_som) print('ARI
Score (K-Means):', ari_kmeans)
print('ARI Score (SOM):', ari_som)
```

OUTPUT

```
Silhouette Score (K-Means): 0.3549055441243031
Silhouette Score (SOM):     0.3128273737018978
Winner (Silhouette):        K-Means (better-defined clusters)

ARI Score (K-Means): 0.3912645301058091
ARI Score (SOM):     0.0609522064892904
Winner (ARI):        K-Means (closer to ground truth)
```







RESULT

K-Means clustering outperformed SOM on both Silhouette Score (0.354 vs 0.312) and Adjusted Rand Index (0.391 vs 0.061), indicating KMeans produced better-defined and more accurate clusters for the Tips dataset.